

```

/*
=====
=====
    firestore-sync.js (PUBLIC SITE)
    -----
    -----
    Connects the public website to the same
    Firestore database the admin
    dashboard writes to. When
    CONFIG.USE_FIRESTORE is true (default), the
    homepage no longer calls the YouTube API
    directly – it subscribes to
    real-time Firestore listeners instead,
    so any Add/Edit/Delete made in
    /admin appears on the public site within
    a second or two, with zero
    page reload and zero YouTube API quota
    spent by visitors.

    If CONFIG.USE_FIRESTORE is false (or
    Firebase isn't configured), the
    site silently falls back to the original
    direct-YouTube-fetch flow in
    api.js – nothing breaks for anyone who
    hasn't set up Firebase yet.

=====
===== */

window.HVC = window.HVC || {};

```

```

window.HVC.FirestoreSync = (() => {
  let songsUnsub = null;
  let artistsUnsub = null;
  let newsUnsub = null;

  let latestSongs = [];
  let latestArtists = [];
  let latestNews = [];

  /**
   * Convert a Firestore song document into
   the same shape api.js produces,
   * so app.js's existing render functions
   work unchanged.
   */
  function normalizeSong(doc) {
    const data = doc.data();
    const uploadedAt =
data.uploadedAt?.toDate ?
data.uploadedAt.toDate() : new
Date(data.uploadedAt ||
data.addedAt?.toDate?.() || Date.now());
    return {
      id: doc.id,
      title: data.title || "",
      thumbnail: data.thumbnail || "",
      channelName: data.artistName ||
"Unknown Artist",
      label: data.label || data.artistName
|| "",
      genre: data.genre || "",
      views: data.views || 0,
      likes: data.likes || 0,

```

```

        comments: data.comments || 0,
        formattedDuration:
data.formattedDuration || "0:00",
        uploadedAt,
        url: data.youtubeUrl ||
(data.youtubeVideoId ?
`https://www.youtube.com/watch?
v=${data.youtubeVideoId}` : "#"),
        featured: !!data.featured,
        trending: !!data.trending,
        manualRank: (data.rankOverride !==
undefined && data.rankOverride !== null) ?
data.rankOverride : null,
        weeklyRank: data.weeklyRank || null,
    };
}

function normalizeArtist(doc) {
    const data = doc.data();
    return {
        id: doc.id,
        name: data.name || "",
        avatar: data.avatar || "",
        bio: data.bio || "",
        channelUrl: data.channelUrl || "",
        instagram: data.instagram || "",
        monthlyViews: data.monthlyViews || 0,
        verified: !!data.verified,
    };
}

function normalizeNews(doc) {
    const data = doc.data();
    const publishedAt =

```

```

data.publishedAt?.toDate ?
data.publishedAt.toDate() : new
Date(data.publishedAt || Date.now());
    return {
        id: doc.id,
        title: data.title || "",
        content: data.content || "",
        thumbnail: data.thumbnail || "",
        author: data.author || "",
        category: data.category || "General",
        status: data.status || "published",
        publishedAt,
    };
}

/**
 * Start real-time listeners. Calls
 * onUpdate() every time ANY collection
 * changes, with the latest { songs,
 * artists, news } snapshot.
 */
function start(onUpdate) {
    if (!window.HVC.db) {
        console.warn("Firestore not
initialized – check firebase-config.js");
        return;
    }

    songsUnsub =
window.HVC.db.collection("songs").onSnapsho
t(
    (snap) => {
        latestSongs =
snap.docs.map(normalizeSong);

```

```

        onUpdate({ songs: latestSongs,
artists: latestArtists, news: latestNews
});

    },
    (err) => console.error("Firestore
songs listener error:", err)
);

artistsUnsub =
window.HVC.db.collection("artists").onSnapsh
hot(
    (snap) => {
        latestArtists =
snap.docs.map(normalizeArtist);
        onUpdate({ songs: latestSongs,
artists: latestArtists, news: latestNews
});
    },
    (err) => console.error("Firestore
artists listener error:", err)
);

newsUnsub =
window.HVC.db.collection("news")
    .where("status", "==", "published")
    .onSnapshot(
        (snap) => {
            latestNews =
snap.docs.map(normalizeNews).sort((a, b) =>
b.publishedAt - a.publishedAt);
            onUpdate({ songs: latestSongs,
artists: latestArtists, news: latestNews
});
        },

```

```
      (err) => console.error("Firestore  
news listener error:", err)  
    );  
  }  
  
  function stop() {  
    if (songsUnsub) songsUnsub();  
    if (artistsUnsub) artistsUnsub();  
    if (newsUnsub) newsUnsub();  
  }  
  
  return { start, stop };  
})();
```